

```

indisvalid      | t
pg_get_indexdef | CREATE INDEX idx_post_created_on ON posts USING hash
(created_on)

```

The `indisunique` column is set to true if the index has been created with the `UNIQUE` clause, as it happens for the primary key index. `indisvalid` is a boolean value that indicates whether the index is usable or not (as you will see later on, you can decide to disable an index for any reason). Since you can cluster a table against an index – that is, you can sort the table depending on a specific index – `indisclustered` indicates whether the table is clustered against the specific index.

The `pg_get_indexdef()` special function provides a textual representation of the `CREATE INDEX` statement used to produce every index and can be very useful to decode and learn how to build complex indexes.

Therefore, either using the `psql \d` command or querying `pg_index`, you can get details about existing indexes and their status.

Dropping an index

In order to discard an index, you need to use the `DROP INDEX` statement, which looks like the following:

```

DROP INDEX [ CONCURRENTLY ] [ IF EXISTS ] name [, ...] [ CASCADE |
RESTRICT ]

```

The statement accepts the name of the index and can drop more than one index at the same time if you specify multiple names on the same statement.

The `CONCURRENTLY` clause prevents the command from acquiring an exclusive lock on the underlying table, preventing other queries from accessing the table until the index has been dropped. Note, this clause cannot always be used; for example, it cannot be used within an explicit transaction.

The `CASCADE` option drops the index and all other objects that depend on the index (e.g., a table constraint), while the `RESTRICT` option is its counterpart and prevents the index from being dropped if any object still insists on the index. The `RESTRICT` clause is the default.

Lastly, the `IF EXISTS` option allows the command to gracefully abort if the index has already been dropped.

Invalidating an index

It is possible to invalidate an index, which is a way to tell PostgreSQL to not consider that index at all without dropping the index and building it again. This can be useful in situations where you are studying your cluster's behavior and want to force the optimizer to choose another path to access the data that does not include a specific index, or it can be necessary when there is a problem with an index.

In order to invalidate an index, you have to directly manipulate the `pg_index` system catalog to set the `indisvalid` attribute to false. For example, in order to suspend the usage of the `idx_author_created_on` index, you have to do an update against `pg_index`, as follows:

```
forumdb=# UPDATE pg_index SET indisvalid = false
          WHERE indexrelid = ( SELECT oid FROM pg_class
                               WHERE relkind = 'i'
                               AND relname = 'idx_author_created_on' );

UPDATE 1

forumdb=# \d posts
...
Indexes:
    "posts_pkey" PRIMARY KEY, btree (pk)
    "idx_author_created_on" btree (author, created_on) INVALID
    "idx_post_category" btree (category)
    "idx_post_created_on" hash (created_on)
...
```



You need to invalidate an index as an administrator user, even if you are the user that created the index. This is due to the fact that you need to manipulate the system catalog, which is an activity restricted to administrator users only.

As you can see, the index is then marked as `INVALID` to indicate that PostgreSQL will not ever try to consider it for its execution plans. You can, of course, reset the index to its original status, making the same update as the preceding and setting the `indisvalid` column to a true value.

Rebuilding an index

Since an index is detached from the data stored in the table, it is possible that the information within the index gets corrupted or somehow out of date. This is not a normal condition, and it does not happen in day-to-day usage of the database, but faulty storage could lead to such a situation. However, knowing how you can rebuild an index is important knowledge because it helps prevent anomalies and allows you to revalidate indexes that have been kept out of date (because they were not valid).

You can always rebuild an index starting from the data in the underlying table by use of the `REINDEX` command, which looks like:

```
REINDEX [ ( VERBOSE ) ] { INDEX | TABLE | SCHEMA | DATABASE | SYSTEM } [
CONCURRENTLY ] name
```

You can decide to rebuild a single index by means of the `INDEX` argument followed by the name of the index, or you can rebuild all the indexes of a table by means of the `TABLE` argument followed, as you can imagine, by the table name.

Going further, you can rebuild all the indexes of all the tables within a specific schema by means of the `SCHEMA` argument (followed by the name of the schema) or the whole set of indexes of a database using the `DATABASE` argument and the name of the database you want to reindex. Lastly, you can also rebuild indexes on system catalog tables by means of the `SYSTEM` argument.

You can execute `REINDEX` within a transaction block but only for a single index or table, which means only for the `INDEX` and `TABLE` options. All the other forms of the `REINDEX` command cannot be executed in a transaction block.

The `CONCURRENTLY` option prevents the command from acquiring exclusive locks on the underlying table in a way similar to that of building a new index.

The EXPLAIN statement

`EXPLAIN` is the statement that allows you to see how PostgreSQL is going to execute a specific query. You have to pass the statement you want to analyze to `EXPLAIN`, and the execution plan will be shown.

There are a few important things to know before using `EXPLAIN`:

- It will only show the best plan, which is the one with the lowest cost among all the evaluated plans.

- It will not execute the statement you are asking the plan for, at least unless you explicitly ask for its execution. Therefore, the EXPLAIN execution is fast and pretty much constant each time.
- It will present you with all the execution nodes that the executor will use to provide you with the dataset.

Let's see an example of EXPLAIN in action to better understand. Imagine we need to understand the execution plan of the `SELECT * FROM categories` statement. In this case, you need to prefix the statement with the EXPLAIN command, as follows:

```
forumdb=> EXPLAIN SELECT * FROM categories;
               QUERY PLAN
-----
Seq Scan on categories (cost=0.00..1.05 rows=5 width=68)
(1 row)
```

As you can see, the output of EXPLAIN reports the query plan. There is a single execution node, of the `Seq Scan` type, followed by the table against which the node is executed (on `categories`). In the output of the EXPLAIN command, you will find all the types of execution nodes already discussed in the previous sections.

For every node, EXPLAIN will report some more information between parentheses: the cost, the number of rows, and the width. The cost is the amount of effort required to execute the node, and is always expressed as a “startup cost” and a “final cost.” The startup cost is how much work PostgreSQL has to do before it begins executing the node; in the preceding example, the cost is 0, meaning the execution of the node can begin immediately. The final cost is how much effort PostgreSQL has to do to provide the last bit of the dataset – that is, to complete the execution of the node.

The `rows` field indicates how many tuples the node is expected to provide in the final dataset, and is a pure estimation. Being an estimation, the value could be wrong and you have to keep in mind that it will never be zero: when PostgreSQL estimates a very low number of tuples, it always provides 1 as the number of rows.

Lastly, the `width` field indicates how many bits every tuple will occupy, as an average. Essentially, this information is used to estimate the network traffic that the query will produce: in the preceding example, it is possible to estimate 68 bytes per tuple.

Now consider another example, just to get used to the EXPLAIN output; the query changes a little to produce a few more nodes, as follows:

```

forumdb=> EXPLAIN
          SELECT title
          FROM categories ORDER BY description DESC;
          QUERY PLAN
-----
Sort  (cost=1.11..1.12 rows=5 width=64)
  Sort Key: description DESC
  -> Seq Scan on categories  (cost=0.00..1.05 rows=5 width=64)

```

Here, we have two different nodes: the first at the top is the **Sort** node (due to the ORDER BY clause), and the second node is Seq Scan, as in the previous example. Please note that there are three output rows, so how do we determine which rows are nodes and which are not? The first row in the plan is always a node, and the other node rows are indented to the right and have an arrow as a prefix (->). The other lines in the plan provide information about the node they are under; therefore, in the preceding example, the Sort Key row is additional information to the **Sort** node.

Another approach to distinguish node rows from additional information is to consider that every node line has the cost, rows, and width attributes in parentheses.

Once you have discovered the nodes of the query, you have to find out the very first node, which is usually the most indented one, and also the one with the lowest startup cost; in the preceding example, Seq Scan is the first node executed. This node does the very same thing explained in the previous example: it forces the executor to go to the table on the physical storage and retrieve, in sequential order, all the table content. One thing, however, is different in the preceding example: the average width has decreased, and this is due to the fact that the query does not require all the columns of every tuple, only the title one.

Once the **Sequential Scan** node has completed, its output is used as input for the **Sort** node, which performs the desired ORDER BY operation. As you can easily read, the sort key of the original statement is printed to provide you with enough information to understand what the executor will sort data on. The **Sort** node has a startup cost that is greater than (better or pretty much the same as) the previous node's final cost: the sequential scan has a final cost of 1.05 and the sort starts with a cost of 1.11. This emphasizes again how nodes are executed in a pipeline, and also tells you that the sort cannot start before the other node has completed. The **Sort** node has a final cost almost equal to the startup cost, meaning that this node is straightforward for PostgreSQL to be executed.

You can try to execute EXPLAIN on different statements to see how a plan changes, and which nodes can be generated, in order to be used to recognize the nodes and the resource information.

In the following subsections, you will see different options to explain a statement.

EXPLAIN output formats

By default, EXPLAIN provides a text-based output, but it can also provide much more structured outputs in XML, JSON, and YAML. These other formats are not only useful when you have to cope with external tools and applications but can also be useful because they provide more information for tuning a query plan.

You can specify the format you want with the FORMAT option followed by the name of the format, which can be TEXT, XML, JSON, or YAML. Take the following example:

```

forumdb=> EXPLAIN ( FORMAT JSON ) SELECT * FROM categories;
          QUERY PLAN
-----
[
  {
    "Plan": {
      "Node Type": "Seq Scan",
      "Parallel Aware": false,
      "Async Capable": false,
      "Relation Name": "categories",
      "Alias": "categories",
      "Startup Cost": 0.00,
      "Total Cost": 1.05,
      "Plan Rows": 5,
      "Plan Width": 68
    }
  }
]

(1 row)

```

As you can see, the JSON format provides not only a different structure to the query plan but also a different and more rich set of information. For example, from the preceding example, we can see that the query has been executed in a nonparallel mode ("Parallel Aware": false,).

If you need to parse the EXPLAIN output with an application or tool, you should stick to one of the structured formats, not the default text one.

EXPLAIN ANALYZE

The ANALYZE mode of EXPLAIN enhances the command by effectively running the query to explain. Therefore, the command does a double task: it prints out the best plan to execute the query and it runs the query, also reporting back some statistical information.

To better understand the concept, consider the output of EXPLAIN ANALYZE compared to the output of a plain EXPLAIN command:

```
forumdb=> EXPLAIN SELECT * FROM posts;
               QUERY PLAN
-----
Seq Scan on posts (cost=0.00..31.04 rows=1004 width=71)
(1 row)

forumdb=> EXPLAIN ANALYZE SELECT * FROM posts;
               QUERY PLAN
-----
Seq Scan on posts (cost=0.00..31.04 rows=1004 width=71) (actual
time=0.006..0.101 rows=1004 loops=1)
Planning Time: 0.052 ms
Execution Time: 0.163 ms
(3 rows)
```

The output of the EXPLAIN ANALYZE command is enhanced by the “actual” part of every node: the executor reports back how the execution of the node went exactly. Therefore, while EXPLAIN can only estimate the costs of a node, the EXPLAIN ANALYZE provides feedback on the execution time (expressed in milliseconds), the effective number of rows, and how many times a node has been executed (loops).

The node time is expressed, similarly to the cost, in a startup time and a final time, which is the time taken for the node to complete its execution. Therefore, in the preceding example, PostgreSQL took 0.006 milliseconds to “warm up” and completed the query execution in 0.101 milliseconds, so the node required 0.107 milliseconds to complete its job.

Another important piece of information in the actual part of the node output is the number of rows (i.e., how many tuples the node has produced), in a similar way to the estimation of a plain EXPLAIN. Why is there a need to again report the number of rows obtained by the node? Remember that EXPLAIN estimates, while EXPLAIN ANALYZE provides you with the effective number of tuples obtained by a node. When these two values are really different, by an order of magnitude or more, PostgreSQL is not able to correctly estimate the size of the result set and hence the best plan to choose to access the data – meaning it may choose a non-optimal access method.

Last, in the actual output, there is the number of loops, that is, the number the very same node has executed. Usually this number is 1, meaning the node has been executed only once, but in the case that the node belongs to a subquery, the value could be greater than 1. The timing and number of rows are related to a single loop execution; therefore, in order to get the final values, you need to multiply rows and time by loops.

At the very end of the command output, EXPLAIN ANALYZE provides overall time information, which includes the planning time, which is the time the optimizer has spent producing the best candidate access plan, and the execution time, which is the total time spent running the query (excluding the parsing and planning time).

Therefore, the preceding example took 0.163 milliseconds to “fetch data” and 0.052 milliseconds in deciding how to fetch the data, and the total time the query took was 0.219 milliseconds (a little more than $0.052 + 0.163$ because some time is spent on resource allocation).



When the data to access is very small, the planning time takes longer than the execution time.

The execution time also includes time spent running BEFORE triggers, while AFTER triggers are not counted because their function is executed once the plan has completed.

The planning time, similarly, accounts only for the time spent in producing the best access plan, not the time required to process rules and writing of the statement, as well as parsing.



EXPLAIN ANALYZE always executes the query you want to analyze; therefore, in order to avoid side effects, you should wrap EXPLAIN ANALYZE in a transaction and roll back the work once the analysis is complete.

EXPLAIN ANALYZE can also be invoked by passing ANALYZE as an option to EXPLAIN, as follows:

```
forumdb=> EXPLAIN ( ANALYZE ) SELECT * FROM categories ORDER BY title
DESC;
```

The option form of EXPLAIN ANALYZE is handy when you want to add other options to EXPLAIN, as shown in the following subsection.

EXPLAIN options

EXPLAIN provides a rich set of options, most of which can only be used in the ANALYZE form. All of the EXPLAIN options presented in this section are boolean, which means they can be turned on and off but nothing more.

The VERBOSE option allows every node to report more detailed information, such as the list of the output columns, even when not specified. For example, even if the query does not explicitly ask for the list of columns, note how, thanks to VERBOSE, you can find out which columns a node will provide to the output dataset:

```
forumdb=> EXPLAIN (VERBOSE on) SELECT * FROM categories;
               QUERY PLAN
-----
Seq Scan on forum.categories (cost=0.00..1.05 rows=5 width=68)
   Output: pk, title, description
(2 rows)
```

The SETTINGS option is used to identify if the query has been planned with a configuration different from the cluster-wide common set of configuration options. Some changes can be applied on a per-session basis; therefore, EXPLAIN is able to report if the session is using a custom setting. As an example, consider changing the `work_mem` parameter that, even if it has nothing to do with the query shown in the following example, is reported by EXPLAIN as a parameter that has been manually changed before the query was planned:

```
forumdb=> SHOW work_mem;
work_mem
-----
4MB
(1 row)
```

```

forumdb=> SET work_mem TO '32MB';
SET
forumdb=> EXPLAIN (SETTINGS on) SELECT * FROM posts ORDER BY created_on
DESC;

               QUERY PLAN
-----
Sort  (cost=164403.31..166903.32 rows=1000004 width=75)
   Sort Key: created_on DESC
   -> Seq Scan on posts  (cost=0.00..20309.04 rows=1000004 width=75)
Settings: work_mem = '32MB'
(4 rows)

```

Thanks to the `SETTINGS` option, it is possible to understand if a query plan has been produced with a custom configuration of certain parameters and, therefore, better understand if such parameters have implications on the query plan.

The `COSTS` option, which is turned on by default, shows the costs part of a node. As an example, turning it off removes the startup and final costs, as well as the average width and the number of rows:

```

forumdb=> EXPLAIN (COSTS off) SELECT * FROM categories;

               QUERY PLAN
-----
Seq Scan on categories
(1 row)

forumdb=> EXPLAIN (COSTS on) SELECT * FROM categories;

               QUERY PLAN
-----
Seq Scan on categories  (cost=0.00..1.05 rows=5 width=68)

(1 row)

```

The `TIMING` option, which is on by default, shows the effective execution time when `EXPLAIN` is invoked with `ANALYZE`. In other words, setting `TIMING` to off means that the output of `EXPLAIN` will not show the time of the node execution.